UNIVERSITATEA DIN BUCUREȘTI

FACULTATEA DE MATEMATICĂ ȘI INFORMATICĂ

SPECIALIZAREA INFORMATICĂ

Disertație

SOFTWARE APPLICATION FOR OS MAINTENANCE

Absolvent
Zăbavă Claudiu-Alexandru

Coordonator științific
Conf.dr. Mihai Prunescu

București, iunie 2025

(Acest document se leagă împreună cu lucrarea de disertație)

Declarație

Subsemnatul/Subsemnata ZĂBAVĂ CLAUDIU-ALEXANDRU, candidat la examenul de disertație, sesiunea IUNIE-IULIE 2025, la Facultatea de Matematică şi Informatică, programul de studii Baze de date și tehnologii software, declar pe propria răspundere că lucrarea de faţă este rezultatul muncii mele, pe baza cercetărilor mele și pe baza informaţiilor obţinute din surse care au fost citate şi indicate, conform normelor etice, în note și în bibliografie. Declar că nu am folosit în mod tacit sau ilegal munca altora și că nici o parte din teză nu încalcă drepturile de proprietate intelectuală ale altcuiva, persoană fizică sau juridică. Declar că lucrarea nu a mai fost prezentată sub această formă vreunei instituții de învățământ superior în vederea obţinerii unui grad sau titlu ştiinţific ori didactic.

Data	Semnătura
17.06.2025	

Rezumat

Pentru a răspunde unei probleme comune a utilizatorilor sistemului de operare Windows, anume scăderea performanței sistemului în timp din cauza acumulării de fișiere redundante, aplicații inutile sau neactualizate și gestionării ineficiente a resurselor, această lucrare sugerează proiectarea și implementarea unei aplicații software avansate dedicate întreținerii și optimizării sistemului.

Prin identificarea fișierelor sau aplicatiilor mari si nefolosite de foarte mult timp sau a fisierelor duplicate, curățarea fișierelor temporare și neesențiale, monitorizarea parametrilor de performanță hardware și controlul actualizărilor driverelor sistemului de operare pentru componentele hardware, proiectul meu de disertație își propune să dezvolte un instrument ce poate oferi suport contextual și personalizat utilizatorului, având o interfață modern și intuitivă, care aduna toate aceste funcționalități pentru utilizator într-un singur loc.

Abstract

To address a common problem of Windows operating system users, namely the decrease in system performance over time due to the accumulation of redundant files, unnecessary or outdated applications and inefficient resource management, this paper suggests the design and implementation of an advanced software application dedicated to system maintenance and optimization.

By identifying large or duplicate files, cleaning temporary and non-essential files, monitoring hardware performance parameters (CPU, RAM, GPU and storage space) and controlling operating system and hardware driver updates, my dissertation project aims to develop a tool that can provide contextual and personalized support to the user, with a modern and intuitive interface, gathering all these features in a single modular application for the user.

Cuprins

1 Introducere **5**

1.1 Motivație 5

1.2 Scopul aplicatiei 6

1.3 Subdomeniul şi tipul lucrării 6

1.4 Arhitectura aplicației 6

1.4.1 Scanarea și ștergerea fișierelor reziduale 7

1.4.2 Filtrarea fișierelor duplicate sau nefolosite 7

1.4.3 Monitorizarea resurselor sistemului 7

1.4.4 Actualizarea sistemului și a driverelor 7

1.4.5 Interfața grafică 8

1.5 Aplicații similare 8

2 Scanarea și ștergerea fișierelor reziduale **10**

2.1 Tipurile de fișiere vizate 10

2.2 Scriptul pentru Fișiere Temporare 11

2.2.1 Functia `getTargets()` 12

2.2.2 Functia `scanTempFiles()` 12

2.2.3 Functia `cleanTempFiles()` 13

2.3 Scriptul pentru Browser Caches 13

2.4 Scriptul pentru Memory Dumps 14

2.5 Scriptul pentru System Error Logs 14

3 Filtrarea fișierelor duplicate sau nefolosite **15**

3.1 Filtrarea fișierelor duplicate 15

3.1.1 Funcția `findDuplicatesByContent()` 15

3.1.2 Funcția `findDuplicatesByName()` 16

3.2 Filtrarea fișierelor nefolosite 16

4 Monitorizarea resurselor sistemului **18**

4.1 Monitorizarea CPU 18

4.2 Monitorizarea GPU 19

4.3 Monitorizarea RAM 20
4.4 Monitorizarea Hard Disk 20

5 Actualizarea sistemului și a driverelor 22
5.1 Driverele pentru GPU 22
5.2 Selectarea driverelor Windows ce necesită actualizări 23

6 Interfața grafică 25
6.1 Panoul *Cleaner* 25
6.2 Panoul *Performance* 28
6.3 Panoul *Updates* 28
6.4 Fereastra pop-up pentru acces rapid 29

7 Concluzii 30
7.1 Perspective și îmbunătățiri 30
7.2 Concluzie finală 30

Bibliografie 31

Capitolul 1

Introducere

1.1 Motivație

Odată cu avansarea rapidă a tehnologiei informației în zilele noastre, sistemele de operare devin instrumente necesare pentru îndeplinirea sarcinilor zilnice, atât în mediu personal, cât și profesional. Una dintre cele mai populare interfețe dintre utilizator și resursele hardware ale computerului este sistemul de operare Windows, care este utilizat și distribuit pe scară largă în întreaga lume. Cu toate acestea, pe măsură ce trece timpul, există o scădere vizibilă a performanței acestor sisteme, care poate fi atribuită unei serii de probleme, inclusiv acumularea de fișiere temporare și inutile, versiuni depășite ale driverelor sau fragmentarea resurselor de stocare.

Din acest motiv, dezvoltarea unui instrument software care poate oferi o soluție coerentă și eficientă pentru întreținerea, optimizarea și monitorizarea sistemului de operare este extrem de urgentă.

Premisa fundamentală a studiului este că un computer de înaltă performanță depinde nu doar de componentele sale hardware, ci și, într-o măsură semnificativă, de calitatea sistemului de operare și al software-ului utilizat pentru gestionarea și întreținerea resurselor.

Am fost determinat să aleg acest subiect deoarece am un interes personal puternic pentru ingineria sistemelor informatice și întreținerea software-ului și doresc să aplic o abordare bazată pe aplicații pentru a ajuta utilizatorul obișnuit de computer să aibă o experiență excepțională, să obțină maximul de performanță de la computer și de la sistemul său de operare.

În concluzie, tema lucrării este susținută de aplicabilitatea și actualitatea sa, precum și de valoarea adăugată pe care o aduce domeniului mentenanței sistemelor informatice personale.

1.2 Scopul aplicatiei

Devine imperativ să se mențină funcționarea optimă a sistemului de operare într-o situație în care utilizarea extensivă a sistemelor informatice este crucială atât în contexte personale, cât și profesionale. Pentru a depăși această dificultate, teza mea creează un program software pentru întreținerea sistemului Windows[1], care se concentrează pe ștergerea eficientă a fișierelor inutile, monitorizarea performanței hardware-ului și controlul actualizărilor pentru driverele de baza ale computerului.

1.3 Subdomeniul şi tipul lucrării

Proiectul meu de absolvire, specific domeniului Tehnologiilor Software cât și Administrației Sistemelor Informatice, se poate clasifică drept aplicație software de tip utilitar. În acest proiect, sunt integrate concepte din diverse subdomenii conexe, inclusiv programare avansată cu accent pe limbajul Java[7], optimizarea și monitorizarea resurselor hardware, interfețe grafice moderne și ușor de utilizat pentru aplicații desktop dezvoltate cu framework-ul JavaFX[6] și aspecte emergente de Web-Scraping cu accent pe limbajul Python. Se urmăresc următoarele competențe: interacțiunea om-calculator, manipularea sistemului de fișiere, dezvoltarea de API-uri și biblioteci externe, proiectarea arhitecturii aplicațiilor desktop și monitorizarea resurselor hardware.

1.4 Arhitectura aplicației

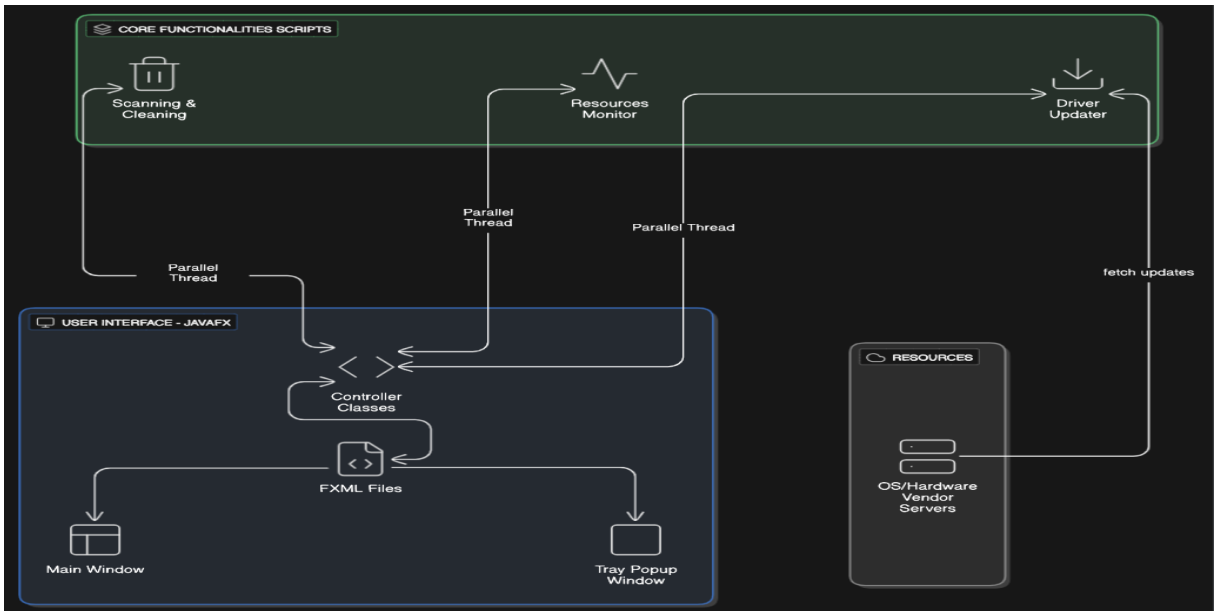

Figura 1.1: Arhitectura aplicatiei

Proiectul de disertație poate fi împărțit conceptual în cinci părți principale pentru a înțelege mai bine funcționalitatea și modularitatea aplicației implementate în cadrul acestuia. Fiecare componentă contribuie în mod esențial la optimizarea performanței sistemului și îmbunătățirea experienței utilizatorului prin automatizare, asistență și monitorizare continuă.

Fiecare dintre aceste secțiuni va fi examinată în detaliu din punct de vedere teoretic și tehnic, începând cu **Capitolul 2**.

1.4.1 Scanarea și ștergerea fișierelor reziduale

Această parte este responsabilă de localizarea și eliminarea fișierelor inutile, redundante sau tranzitorii care pot cauza încetinirea sistemului prin ocuparea unor volume mari din spațiu de stocare. Folosind funcții ale bibliotecii Java pentru a găsi fișiere inutile, scanarea se efectuează prin parcurgerea metodică a câtorva locații importante ale sistemului de operare.

1.4.2 Filtrarea fișierelor duplicate sau nefolosite

Această parte este responsabilă de localizarea, filtrarea și posibil eliminarea a fișierelor care nu au mai fost accesate pe o perioada mare de timp. Pe lângă asta, aplicația se va ocupă și de filtrarea fișierelor duplicate din locațiile specificate de utilizator, oferindu-i posibilitatea de a le șterge.

1.4.3 Monitorizarea resurselor sistemului

Scopul acestui modul este de a colecta date despre utilizarea procesorului, memoria RAM, spațiul pe disc și alte resurse critice ale sistemului în timp real. Monitorizarea se bazează pe utilizarea bibliotecilor din scripturile Python pentru a interoga sistemul de operare.

1.4.4 Actualizarea sistemului și a driverelor

Un alt modul crucial este cel care se ocupă de verificarea și, dacă este posibil, de inițierea actualizărilor driverelor sistemului de operare de pe dispozitiv. Anumite comenzi și scripturi sunt folosite pentru a opri sau porni cautarea de actualizari pentru driverele Microsoft, sau pentru a cauta , pe baza componentelor hardware precum GPU, drivere și actualizări de drivere pe site-urile producătorilor.

1.4.5 Interfața grafică

Framework-ul JavaFX a fost utilizat pentru dezvoltarea interfeței, împreună cu clase Controller pentru a oferi legătura logică între elementele UI și funcționalitatile oferite de scripturile din back-end, precum și cu fișiere FXML pentru a specifica structura vizuală.

Interfata grafica consta in 2 ferestre:

- **Fereastra principală**, care acționează ca punct central al aplicației și oferă acces la fiecare modul disponibil;
- **Fereastra de tip popup**, utilizata pentru acces rapid la panoul de scanare și curățare sau la panoul de monitorizare.

Datorită designului său modular, aplicația poate fi dezvoltată treptat, iar funcționalitățile sale pot fi adăugate cu ușurință în iterații ulterioare.

1.5 Aplicații similare

Întreținerea sistemului de operare a fost și va fii mereu un topic important pentru pasionații de computere. Din acest motiv, există aplicații care au fost și încă sunt dezvoltate pentru a ține pasul cu necesitățile utilizatorilor în ceea ce privește mentenanță sistemelor.

Acestea m-au inspirat m-au inspirat și pe mine să dezvolt propria mea aplicație cu mici adaosuri sau îmbunătățiri.Câteva exemple demne de menționat sunt:

- **CCleaner**[1]:

  Cu capabilități precum controlul pornirii aplicațiilor, ștergerea registrului Windows și eliminarea fișierelor temporare, CCleaner este unul dintre cele mai utilizate programe de curățare a fișierelor rămase.

  Prezintă o interfață ușor de utilizat și capacități eficiente de curățare. Cu toate acestea, CCleaner a fost criticat pentru instalarea de componente suplimentare nedorite și reclame și a fost subiectul mai multor probleme de confidențialitate.

  Acest lucru a împins mai multiutilizatori în căutarea unei aplicații înlocuitoare.

- **Advanced SystemCare**[2]:

  Advanced SystemCare, aplicație software creată de IObit, oferă o abordare semi-automată a optimizării sistemului, care include curățarea log-urilor si a cache-ului,

[1]Tool-ul CCleaner: https://www.ccleaner.com/ccleaner
[2]Tool-ul Advanced SystemCare: https://www.iobit.com/en/advancedsystemcarefree.php

detectarea programelor malware, actualizări ale driverelor și accelerarea rețelei. Este simplu de utilizat și are un aspect plăcut. Cu toate acestea, doar ediția premium „Pro” are acces la unele dintre funcțiile sale cheie. Alte reclamații frecvente includ „bloatware” și alerte excesive.

- **AVG TuneUp**[3]:

  AVG Technologies propune o soluție care utilizează o strategie de „mentenanță predictivă” ce recomandă optimizări automate în funcție de utilizarea sistemului. Pentru a economisi resurse, are și capabilități precum posibilitatea de a hiberna aplicațiile rareori utilizate.

Prin eliminarea dezavantajelor exemplelor menționate anterior, soluția creată în această lucrare urmărește să combine punctele forte ale aplicațiilor menționate mai sus, cum ar fi: capacitatea de a șterge jurnalele și registrele de sistem, memoria cache și fișierele temporare, precum și de a scana și elimina aplicațiile neutilizate și de a le controla; dar și adăugarea de noi funcționalități:

- Monitorizare în timp real a performanței sistemului, cu valori procentuale pentru CPU, GPU, RAM și memorie disponibilă;
- Fara alte ocmponente sau servicii suplimentare care sa deranjeze sau sa pericliteze confidentialitatea utilizatorului. Aplicatia nu salveaza informatii personale, ci strict date legate de actualizari pentru sistemul de operare sua pentru drivere, acestea fiind sterse dupa fiecare sesiune.
- Controlul actualizărilor driverelor sistemului de operare , cu sugestii automate pentru menținerea compatibilității;

Astfel, aplicația rezultată este un instrument intuitiv și extensibl, un agregator de funcționalități multiple.

[3]Tool-ul AVG TuneUp: https://www.avg.com/en-us/avg-pctuneup

Capitolul 2

Scanarea și ștergerea fișierelor reziduale

Un element cheie al aplicației dezvoltate de catre mine în cadrul acestei lucrări este tehnica de identificare și eliminare a fișierelor reziduale. Această caracteristică își propune să reducă fragmentarea resurselor, să crească viteza sistemului și să elibereze spațiu de stocare. Acest tip de fisiere se acumulează de obicei ca urmare a utilizării zilnice a computerului și al numeroaselor aplicații, iar în absența unui mecanism automat de întreținere, acestea pot afecta performanța generală a sistemului.

Componentele software utilizate pentru examinarea și curățarea acestor fișiere sunt descrise în detaliu în acest capitol.

2.1 Tipurile de fișiere vizate

Mai jos, am grupat principalele categorii de date reziduale vizate pentru scanare și curățare, acestea acoperind : fișiere temporare de sistem, loguri de erori, fișiere cache ale browserelor utilizatorului, foldere abandonate și fișiere de tip memory dump generate automat de sistemul de operare în momentul unui crash.

Categorie	Descriere
Temporary Files	Fișiere temporare generate în APPDATA sau SystemRoot de către aplicațiile utilizatorului
Browser caches	Fișiere reziduale ce apar în urma folosirii browser-ului pe perioade îndelungate și pentru diferite activități.
Memory Dumps	Fișiere create de sistemul de operare Windows, ce retine o copie a conținutului memoriei RAM in momentul unei erori critice.
System Error Logss	Log-uri și rapoarte cu date despre erori sau crash-uri ale aplicațiilor de sistem.

Tabela 2.1: Categoriile de fisiere scanate si sterse la curatare

Deși în teorie ar trebuie eliminate automat după utilizarea aplicațiilor, în practică acestea rămân frecvent pe disc, ocupând într-un mod inutil spațiul de stocare.

2.2 Scriptul pentru Fișiere Temporare

Numeroase procese și programe ale sistemului de operare Windows creează fișiere temporare pentru a stoca cache-uri, date intermediare sau rezultate temporare.

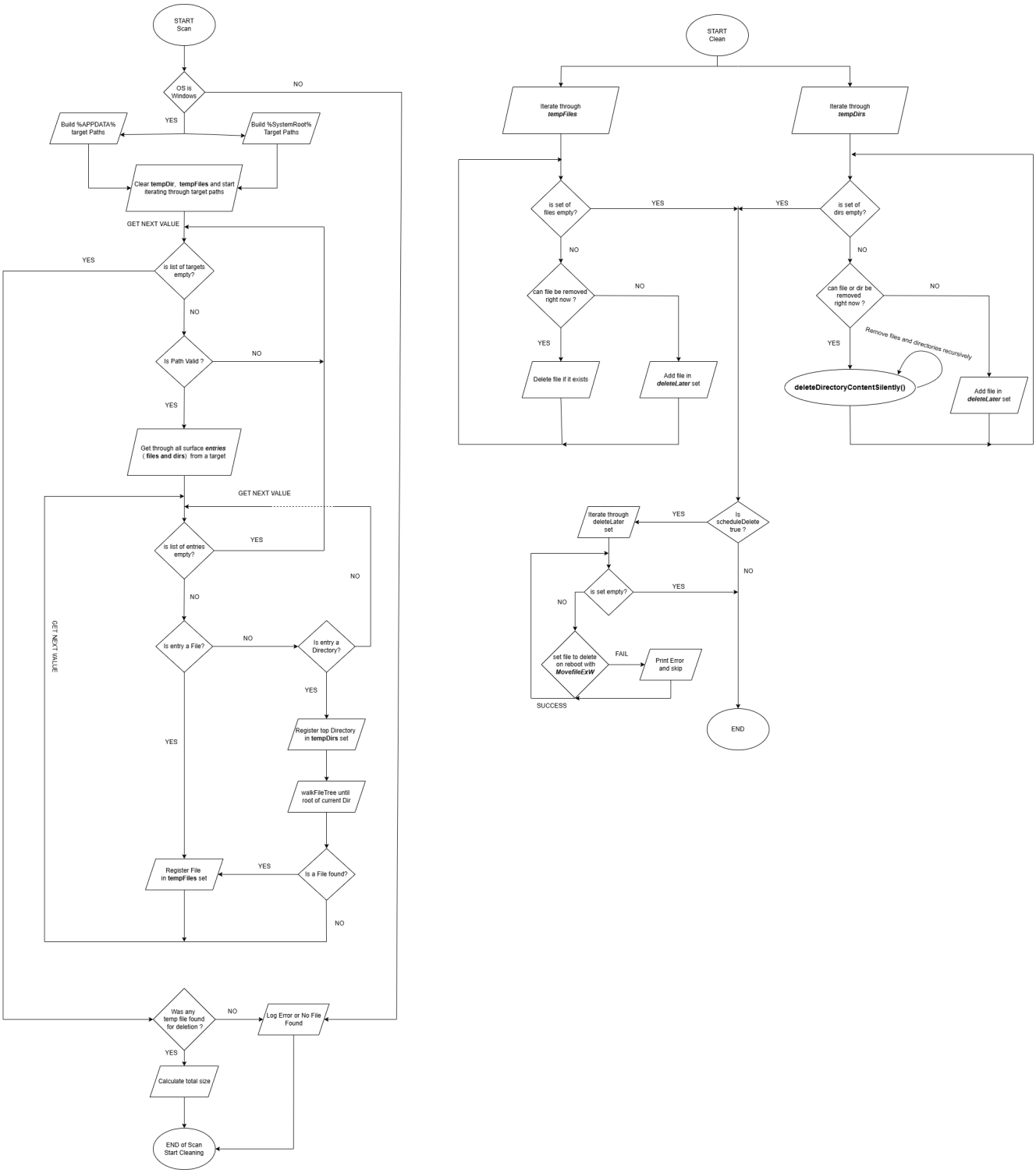

Figura 2.1: Flow-ul sistemului de scanare și curățare

Găsirea și eliminarea acestor fișiere a fost concepută pentru a fi automatizată de

scriptul ***TempFilesScript***.

Acest script Java conține funcții de scanare și curățare pe baza cărora au fost concepute și celelalte scripturi din acest capitol, cu mici ajustări în funcție de tipul de date vizat. Prin urmare, o sa le descriem în detaliu pe fiecare în aceasta secțiune.

Identificarea tuturor locațiilor comune pentru fișierele temporare, calcularea dimensiunii lor totale și, cu acordul utilizatorului, pornirea procedurii de curățare sunt toate gestionate în cadrul acestei clase Java. Scriptul este compatibil numai cu sistemele Windows și utilizează anumite API-uri native *Windows*, precum*MoveFileExW* pentru a traversa structurile de foldere în căutarea, scanarea și ștergerea fișierelor temporare.

2.2.1 Functia `getTargets()`

Această funcție este folosită pentru a constru automat căile către folderele utilizate pentru stocarea fișierelor temporare utilizând variabile de sistem precum `APPDATA` și `SystemRoot`. În sistemele de operare Windows, fișierele temporare adesea se găsesc în următoarele lcoatii:

- *`AppData / Local / Temp`* – folderul principal atunci când vine vorba de fișierele temporare create de aplicațiile utilizatorului.
- *`AppData / LocalLow / Temp`* – spațiu adesea utilizat de aplicații cu privilegii reduse sau în moduri de execuție restricționate.
- *`Windows / Temp`* – directorul global la nivelul sistemului de operare pentru fișiere temporare generate de serviciile Windows.

2.2.2 Functia `scanTempFiles()`

Bazată pe o arhitectură hibridă de traversare secvențială și recursivă a sistemului de fișiere, funcția **`scanTempFiles()`** implementează un proces fiabil de inspecție a folderelor. Folosind **`DirectoryStream`** (importat din biblioteca *java.nio.file*) o soluție performanta pentru traversarea liniară a directoarelor fără a încărca întreaga structură în memorie, se inițiază o buclă efectivă prin conținutul directoarelor țintă indicate mai sus.

Vizitând fiecare din aceste directoare,se va inița o parcurgere recursiva prin **`Files.walkFileTree`** împreună cu un **`SimpleFileVisitor`**, care asigură scanarea și găsirea oricărui fișier din arborii de foldere, inclusiv în subniveluri.

Calea către fiecare fișier scanat este salvata temporar într-o colectie de tip *Set* ce va fii folosit ulterior in procesul de curățare. Se vor retine și căile directoarelor părinte a tuturor fișierelor temporare găsite.

2.2.3 Functia `cleanTempFiles()`

Aceasta funcție folosește API-uri avansate ***`java.nio.file`*** pentru a elimina fișiere și directoare temporare din sistem într-un mod tolerant la erori.

Folosind metoda Files-deleteIfExists, procedură care începe prin iterarea colecției ***`tempFiles`***, verificând existența fiecărui fișier înainte de a-l elimina. Adăugarea fișierului la o listă de ștergere întârziată (***`deleteLater`***) și activarea unui semnal logic (***`scheduleDelete`***) pentru a începe curățarea programată la repornire sunt două modalități de a aborda erorile de ștergere, care sunt de obicei cauzate de *deadlocks* (handle-uri deținute de un proces) sau de lipsa permisiunilor.

Apelata în metodă **`cleanTempFiles()`**, funcția **`deleteDirectoryContentsSilently()`** se ocupă de folderele găsite în lcoatia țintă după scanare, utilizând Files pentru a procesa directoarele din tempDirs. Această navigeaza recursiv în structură și elimină fișierele găsite, combinând procedura walkFileTree cu un SimpleFileVisitor personalizat. Cu excepția directorului rădăcină, fiecare subdirector este șters după ce a fost vizualizat complet.

Atunci când anumite fișiere sunt blocate, este permisă o curățare întârziată utilizând API-ul nativ *MoveFileExW* și flag-ul MOVEFILE-DELAY-UNTIL-REBOOT(0x00000004), care plasează în registrul de sistem instrucțiuni de eliminare definitivă a fișierelor la următorul ciclu de boot.

2.3 Scriptul pentru Browser Caches

Pentru a adresa o sursă semnificativă de utilizare inutilă a spațiului pe disc și pentru a contribui la protejarea confidențialității utilizatorilor, scriptul BrowserCacheScript.java urmărește să detecteze și să șterge automat fișierele cache produse de browserele web populare (cum ar fi *Opera*, *Chrome* etc.) pe Windows.

Funcția de scanare a acestui script se bazează pe cea prezentată anterior în **Figura 2.1** din secțiunea anterioară. Majoritatea browserelor moderne își salvează memoria cache în directoare situate în `APPDATA, LOCALAPPDATA` sau `ProgramData`. Scriptul folosește **`Files.walkFileTree`** pentru a construi o listă de foldere țintă pe baza acestor locații, care sunt apoi căutate recursiv folosind o metodă de traversare în profunzime. Evaluarea dimensiunii cache este salvata in variabila `totalSize`, apoi transformată într-un format lizibil folosind funcția **`humanReadableByteCount()`** pentru conversie din bytes în MB sau GB.

Funcția de curățare începe cu o încercare sincronă de a elimina fiecare fișier separat și folosește același mecanism de tip fail-safe ca in sectiunea anterioara. Fișierele sunt desemnate pentru ștergere la repornire folosind `MoveFileExW` dacă ștergerea directă eșuează din cauza unui deadlock.

2.4 Scriptul pentru Memory Dumps

Când apar erori grave, cum ar fi blocările ecranului albastru sau erorile de program, sistemul de operare Windows creează automat fișiere de imagine a memoriei, care includ instantanee precise ale memoriei *RAM* la momentul incidentului. Deși sunt utile în situații de depanare mai complexe, acestea ocupă frecvent sute de megaocteți sau chiar gigaocteți și, fără o examinare umană, sunt lăsate neutilizate și uitate în locuri comune precum `SystemRoot / MEMORY.DMP` sau `LOCALAPPDATA / CrashDumps`. Acumularea lor duce la o utilizare inutilă a spațiului pe disc, ceea ce afectează în special sistemele cu spațiu de stocare redus.

Scriptul **`MemoryDumpScript.java`** se bazeaza pe o arhitectura de scanare și curățare asemanatoare (folosind Files.walkFileTree, deleteIfExists și soluția nativă `MoveFileExW`), insa se diferentiaza prin vizarea specifică a locațiilor de stocare pentru crash dumps și filtrarea fișierelor pe baza extensiilor specificate (**.dmp**). În plus, spre deosebire de tacticile utilizate pentru memoria cache sau folderele temporare, fișierele descoperite sunt tratate ca entitățide dimensiuni mari și li se acordă prioritate pentru curățarea imediată.

2.5 Scriptul pentru System Error Logs

Windows creează automat jurnale de raportare a erorilor (*WER*) după ce aplicațiile sau componentele sistemului se blochează. Aceste fișiere, care sunt stocate în directoarele `Microsoft/ Windows/ WER` [4] din zona `LOCALAPPDATA` sau `ProgramData`, includ metadate, fișiere XML care conțin statistici și detalii despre excepții. În general, acestea sunt fișiere mai mici decât imaginile de memorie de la memory dumps.

În acest script sunt este folosita funcția `locateErrorLogDirs()` care construiește lista `scanDirs` cu directoarele vizate pentru scanare, iar apoi scanarea se realizează prin metoda **`scanSystemErrorLogs()`**. Curățarea tuturor fișierelor găsite în timpul scanării se executa cu ajutorul funcțiilor **`cleanSystemErrorLogs()`** si **`deleteDirectoryContentsSilently()`**. Arhitectura acestor metode este asemănătoare cu cea prezentata la începutul capitolului, in **Figura 2.1** .

Toate aceste scripturi sunt manipulate de către clasa **`MainController`**. Se folosește conceptul de abstracție asincronă prin utilizarea Task-urilor pentru a rula fiecare sarcina pe un Thread separat. Crearea unui task se realizează prin '**`Task<Void> cleaningTempTask = new Task<>()`**' iar acesta conține instrucțiuni necesare desfășurării activității. Metoda anonimă **`checkIfdone()`** joacă rolul unui *WatchDog*, asigurându-se că fiecare task , respectiv thread, s-a încheiat. Este apelata de fiecare thread inainte sa se inchida, iar în interiorul ei se numără task-urile încheiate cu succes si task-urile încheiate cu eroare, adăugând în variabila `tempMessage` mesajul corespunzător ce se va afișa în UI.

Capitolul 3

Filtrarea fișierelor duplicate sau nefolosite

O cauză semnificativă a aglomerării spațiului de stocare, fișierele duplicate sau insuficient utilizate, au un efect negativ asupra experienței utilizatorului și a performanței sistemului .În cele ce urmează voi descrie modalitatileprin care aplicația implementată rezolva aceste neplăceri.

3.1 Filtrarea fișierelor duplicate

Aplicația identifică și gestionează fișierele duplicate într-un mod sistematic atat prin analiza conținutul fișierelor, dar și prin analiza numelor acestora. Butonul „Selectați directorul" permite utilizatorului să aleagă un director țintă, iar apoi Întreaga logică de identificare a duplicării este gestionată de o clasă specifică numită **`DuplicateFileFinder`**, ce folosește 2 funcții importante:

3.1.1 Funcția `findDuplicatesByContent()`

Funcția urmează un algoritm de filtrare în doi pași. Inițial, fișierele sunt clasificate în funcție de dimensiunea lor în octeți. Această primă etapă permite limitarea spațiului de căutare, deoarece două fișiere de dimensiuni diferite nu pot avea același conținut. Fișierele de aceeași dimensiune sunt apoi supuse unei analize detaliate prin calcularea unei amprente criptografice, și anume un hash **`SHA-256`**. Un flux de tip `InputStream` este utilizat pentru a citi fiecare fișier binar și a calcula această valoare hash secvențial, în blocuri de 4096 de octeți.

Amprentele digitale obținute sunt apoi grupate într-o structură de tip map : `Map<Integer, List<File>>` și orice fișiere cu valori *hash* identice sunt considerate duplicate, path-ul lor fiind stocat sub aceeași cheie în cadrul structurii.

Această structura Map este transmisă ulterior către clasa **`MainController`** pentru a interpreta și integra rezultatele în interfața grafică. Pentru fiecare fișier duplicat identificat, clasa `MainController` generează dinamic un element vizual de `HBox`, grupurile de duplicate fiind marcate de culori asemănătoare.

3.1.2 Funcția `findDuplicatesByName()`

Am luat în calcul și scenariul în care utilizatorul crează sau editează un anumit fișier, adăugând sau stergant anumite părți din conținut în dealungul timpului, astfel că de la filă originală va ajunge să aibe mai multe copii, fiecare cu câte un mic adaos sau modificare. În acest sens, filtrarea după conținut nu va detecta că toate aceste fișiere sunt oarecum duplicate.

Metoda **`findDuplicatesByContent()`** , în adiție la funcția descrisă anterior și în combinație cu un checkbox în UI, permite utilizatorului să filtreze și grupeze fișierele duplicate după gradul de similaritate al numelui.

Pentru a detecta fișiere cu nume similar (cu un grad de similaritate de minim 80 de procente în cazul codului scris de mine) se folosește celebrul algoritm ***Levenshtein Distance*** care măsoară numărul de editări (înserare, ștergere, înlocuire) necesare pentru a transformă un șir de caractere în altul.

$$\mathrm{lev}_{a,b}(i,j) = \begin{cases} \max(i,j) & \text{if } \min(i,j) = 0, \\ \min \begin{cases} \mathrm{lev}_{a,b}(i-1,j) + 1 \\ \mathrm{lev}_{a,b}(i,j-1) + 1 \\ \mathrm{lev}_{a,b}(i-1,j-1) + 1_{(a_i \neq b_j)} \end{cases} & \text{otherwise.} \end{cases}$$

Figura 3.1: Ecuația distantei Levenshtein

Am refolosit aceeași structura de tip map (`Map<Integer, List<File>>`) în combinație cu procedura `Files.walk` pentru a gă și si compară fiecare fisies din directorul ales de către utilizator.

Fișierele care au un nume cel puț în 80% similar, sunt înregistrate în grupuri de liste care mai apoi vor fii inserate în structura de tip Map. Structura este returnata in final spre clasa `MainController` care se ocupa de inserarea fiecărei intrări în UI.

3.2 Filtrarea fișierelor nefolosite

La fel ca în cadrul filtrarii duplicatelor, utilizatorul alege un director tântă care este trimis funcției **`getLastAccessTimes()`**, ce produce o alta structura de tip Map (

`Map<File, LocalDateTime>`) care leagă fișierele (reprezentate de obiecte tip **`File`**) de ultimele ore și date de acces (reprezentate de obiecte **`LocalDateTime`**).

Aceste date sunt obținute prin parcurgerea recursivă a directorului furnizat și a tuturor subdirectoarelor sale folosind funcția `Files.walk`. Un flux de căi (Path) este returnat de metoda `Files.walk;` , aceste căi sunt filtrate pentru a conține doar fișiere normale, excluzând directoarele. Atributul **`lastAccessTime`** este apoi extras pentru a stabili ultima dată și oră de acces pentru fiecare fișier, odată ce atributele sale au fost citite folosind funcția `Files.readAttributes`.

Pentru a permite manipularea suplimentară a datelor, harta este aranjată în ordine inversă accesării, accentuând fișierele accesate cu mult timp în urmă și plasând fișierele accesate cel mai recent la sfârșitul listei.

Structura Map este transmisă ulterior către clasa **`MainController`** pentru a interpreta și integra rezultatele în interfața grafică, la fel ca în cazul fișierelor duplicate.

Atât scriptul pentru fișiere duplicate, cât și cel pentru fișiere nefolosite, oferă și o funcție de ștergere intitulată **`deleteFile()`**.

Cu ajutorul ei și al butoanelor create și integrate în interfață grafică, fiecare fișier listat poate fi șters pe loc (sau la reboot în caz că este blocat) din aplicație.

Capitolul 4

Monitorizarea resurselor sistemului

Aplicația creată de mine utilizează un cadru modular asincron pentru a colecta și afișa datele despre hardware în timp real. Acest sistem se concentrează pe patru componente cruciale ale computerului: HardDisk, RAM, GPU și CPU. Pentru a îndeplini acest obiectiv, m-am folosit de un script *Python* dedicat colectării nivelului de utilizare al GPU-ului, datorită capacităților și bibliotecilor disponibile în acest limbaj de programare, precum **`gpustat`**[5], iar pentru restul componentelor hardware am folosit cat un script Java ce se bazează în procesul de colectare a datelor pe biblioteca **`OSHI`**[3] .

4.1 Monitorizarea CPU

Este realizata prin intermediul a doua scripturi Java:

- Scriptul **`CpuInfoFetcher`** - Numele complet al modelului procesorului, numărul de nuclee fizice și numărul thread-uri se numără printre metadatele pe care acest script le aduna folosind librăria `OSHI` și le transmite mai departe către interfața grafica.

- Scriptul **`cpuUsage`** - Este responsabil de interogarea directă a procesorului, prin metoda **`getSystemCpuLoad()`**, pentru a obține procentajul de utilizare al nucleelor acestuia. Măsurătoarea se realizează pe baza a două snapshot-uri diferite înregistrate la o distanță de 500ms pentru a asigura o estimare cât mai precisă.

In controlerul principal al interfetei grafice, metoda **`setupCpuChart()`** preia datele livrate de `CpuInfoFetcher` la activarea interfeței și creează un layout *PieChart*, inițial gol, ce va afișa procentul de utilizare în timp real. Metoda **`startMonitoring(target)`** inițiază un thread-executor programat (prin ***`newSingleThreadScheduledExecutor()`***), care este responsabil pentru actualizarea dinamică a procentului de utilizare CPU în interfața grafica. Acest executor apelează metoda **`getSystemCpuLoad()`** din scriptul `cpuUsage.java` într-un thread separat și este configurat să se execute la fiecare 5 secunde, actualizand constant valoarea afișată în UI prin funcția **`updatePieChart`**.

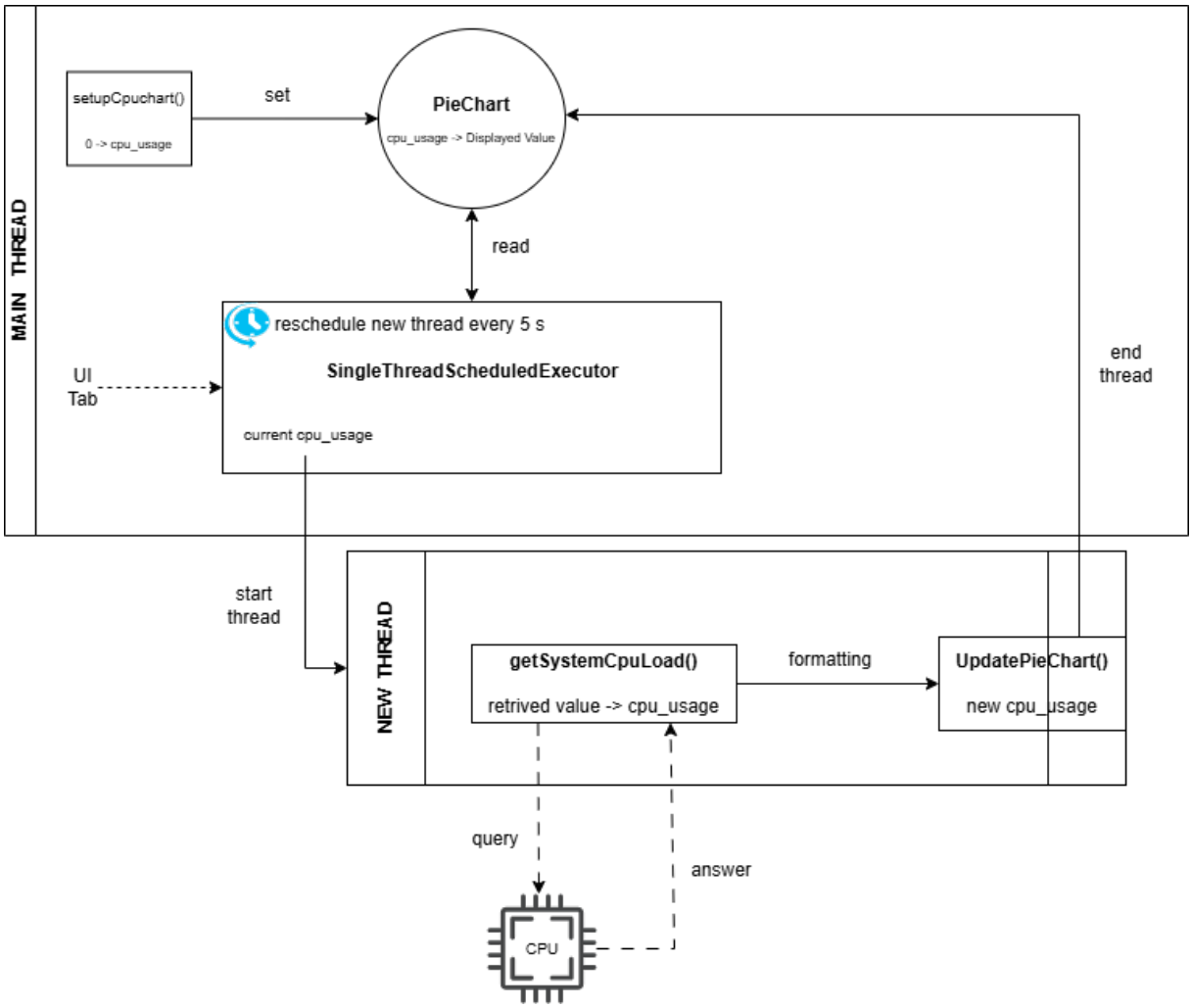

Figura 4.1: Arhitectura monitorizării dinamice a CPU prin thread-uri paralele

4.2 Monitorizarea GPU

Este posibilă datorită unei soluții hibride bazată pe interacțiunea dintre *Java* și *Python* în cadrul modulului de monitorizare a resurselor grafice pentru a recupera, procesa și afișa procentul de utilizare a plăcii grafice (`GPU`) în timp real. Sistemul se bazează pe o arhitectură modulară, în care scriptul **`gpuUsage.py`** apelat din clasa **`MainController`** a aplicației noastre, oferă acces la date vitale despre funcționarea acceleratorului grafic. Acest apel între două limbaje de programare diferite se datorează faptului că Java nu oferă suport nativ complet pentru monitorizarea GPU, în special în sistemele Windows.

Biblioteca *`OSHI`*[3] este utilizată de clasa **`GpuInfoFetcher`** pentru a colecta informații fixe despre GPU-ul principal, cum ar fi numele dispozitivului grafic și memoria video totală. Interfața grafică afișează aceste informații drept metadate. Un script *Python* separat , denumit **`gpuUsage.py`**, care interoghează utilizarea GPU-ului în timp real folosind libraria `gpustat` gestionează monitorizarea activă a procesorului grafic. În scopul de a crea un proces extern menit sa se ocupe de fluxul de monitorizare al aplicației, am ales

sa folosesc un `ProcessBuilder` specific *Java* care orchestrează apelul către acest script creat într-un limbaj de programare diferit de restul aplicației. Un executor dedicat (**`ScheduledExecutorService gpuExecutor`**) rulează scriptul la fiecare 5 secunde, citește procentul de utilizare returnat și actualizează interfața grafica, la fel cum este ilustrat în ***Figura 4.1*** prezentata anterior . În conformitate cu conceptele de izolare a erorilor și de răspuns rapid, această metodă segmentată și asincronă pe thread-uri separate permite o monitorizare GPU eficientă și neîntreruptă.

4.3 Monitorizarea RAM

Scopul acestei sarcini asincrone modularizate este de a oferi utilizatorului o reprezentare vizuală precisă și dinamică a resurselor de memorie volatile.

Biblioteca `OSHI` oferă o interfață prin care se se poate realiza interogarea componentelor hardware pentru mecanismul de colectare a datelor RAM. Pentru a estima performanța generală a subsistemului RAM, la inițializare sunt extrase trei atribute de bază: modelul de memorie (derivat din primul modul fizic detectat), capacitatea totală instalată exprimată în gigaocteți și frecvența de bază exprimată în gigaherți. Aceste date sunt afișate ca metadate descriptive într-o manieră statică în interfața grafică cu utilizatorul.

Din thread-ul principal al aplicației pornește un alt threrad-executor intitulat **`ramExecutor`** prin comanda `ScheduledExecutorService`, configurat să ruleze funcția **`getavailableRam()`** din clasa **`RaminfoFetcher`** la fiecare cinci secunde, pentru a urmări utilizarea memoriei RAM în timp real. În acets proces, stratul de abstractizare hardware este accesat direct pentru a interoga memoria RAM disponibilă. Cantitatea de RAM ocupată este determinată prin calcularea diferenței dintre memoria totală instalată și memoria liberă curentă. Acest rezultat este transmis interfeței grafice ***`JavaFX`*** printr-o metodă thread-safe (folosind `Platform.runLater`), unde este afișat într-o alta diagramă circulară animată. Procentul de utilizare este derivat prin formula `(used - free) / used * 100` .

4.4 Monitorizarea Hard Disk

- Se realizează folosind conceptul de observabilitate periodică aplicat anterior, bazându-se pe colectarea de date privind spațiul total, liber și utilizat. Colectarea și afișarea acestor date se realizează pe un fir de execuție dedicat, care este controlat de un alt thread-executor numit **`diskExecutor`**, la fel ca în cazul componentelor CPU și RAM, pentru a evita blocarea procesului principal al aplicației.

În momentul inițializării interfeței grafice, funcția de de creare a elementelor vizuale este apelată , creând un `PieChart` în starea sa originală cu 100% spațiu liber și 0% consum. În paralel, folosind obiectele ***`HWDiskStore`***, care sunt o componentă a bibliotecii de introspecție a sistemului (*OSHI*), sunt colectate metadate descriptive despre unitatea principală

de stocare, inclusiv modelul fizic al discului. Împreună cu o valoare numerică care reprezintă întreaga capacitate a discului, aceste informații sunt prezentate utilizatorului într-o interfață ce contine campuri de text. Ulterior, diagrama tip *Pie* o sa fie actualizată pe baza valorilor extrase în mod periodic de către funcția **`retrieveDiskData()`** prezentă în scriptul **`DiskInfoFetcher`**, proces similar cu cel descris în *Figura 4.1*.

O caracteristică tehnologică importantă este că, spre deosebire de monitorizarea RAM, care necesită o derivare suplimentară a procentului de utilizare, monitorizarea discului calculează deja direct valorile spațiului utilizat și liber, permițând o actualizare directă și eficientă a graficului. Deoarece agregarea datelor se efectuează incremental pentru toate rădăcinile accesibile, metodă simplifică și scalarea unităților de stocare suplimentare.

Într-o aplicație multi-process, precum cea pe care am implementat-o in cadrul acestei lucrări, gestionarea corectă a firelor de execuție și a proceselor create în timpul rulării este esențială pentru menținerea stabilității, scalabilității și eficienței.

În scenariul monitorizării procesorului grafic, implementarea pornește periodic un proces extern care execută un script *Python*. Acest comportament implică deschiderea repetată a instanțelor de proces, ceea ce poate duce la pierderi de memorie, blocaje sau acumularea de procese orfane (cunoscute și sub numele de procese zombie).

Arhitectura concepută pentru aplicație folosește două strategii complementare pentru a evita aceste rezultate. Controlul asupra fiecărui ciclu de execuție este posibil datorită faptului că am implementat câte un thread-executor distinct per componentă hardware. Astfel, atunci când utilizatorul va închide aplicație, funcția **`shutdownExecutors()`** este apelata, punând capăt fiecărui executor și eliminând orice thread creat de acestea.

În plus, în cazul proceselor lansate pentru a rula scriptul Python de colectare de date, înainte de fiecare iteratie în timpul a 5s se folosește comandă **`gpuProcess.waitFor()`** pentru a asigura închiderea procesului existent imediat după extragerea rezultatului.

Capitolul 5

Actualizarea sistemului și a driverelor

5.1 Driverele pentru GPU

Driverul plăcii video este necesar în special pentru optimizarea subsistemului grafic, compatibilitatea aplicațiilor grafice și performanța generală a sistemului . Folosind tehnici hibride care combină interogări precise către sursele și site -urile oficiale *NVIDIA* cu introspecția sistemului local, aplicația dezvoltată în cadrul acestui proiect de disertație încorporează o abordare automată pentru detectarea și descărcarea celor mai recente versiuni de drivere GPU.

Apăsarea unui buton în interfața grafică pornește întreg procesul de actualizare prin lansarea asincron a unui subproces în cadrul obiectului **`Task<Void> checkGpuUpdatesTask`** din `MainController`. Scriptul (**`scrapper.py`**) este apelat de acest subproces, iar acesta se va ocupă atât de obținerea versiunii actuale ale driverului pentru placă video instalată, dar și de *scraping*[9] pentru colectarea de detalii despre ultimele versiuni disponibile prin obținerea accesului la resursele oficiale *NVIDIA* cu ajutorul API-ului AJAX. Pentru stabilirea ultimei versiuni driver valabile, scriptul executa o interogare indirectă în baza de date NVIDIA, prin accesarea programatică a paginii `Lookup-XML`[1] în cauză. Pe această pagină este furnizată o colecție de elemente **`LookupValue`**, fiecare dintre acestea având numele modelului, identificatorul părintelui și codul numeric al modelului drept caracteristici. Pentru a căuta dinamic un element al cărui câmp de nume se potrivește cu numele plăcii video a utilizatorului, scriptul analizează fișierul XML folosind biblioteca **`xml.etree.ElementTree`**[8].

Pe baza identificatorilor extrași, scriptul formulează o interogare GET către endpoint-ul ***`AjaxDriverService`*** al *NVIDIA*.

Un obiect *JSON* cu un câmp IDS care conține cele mai recente informații despre

[1]Pagina NVIDIA pentru Lookup-XML: https://www.nvidia.com/Download/API/lookupValueSearch.aspx

versiunea driverului (Version și DownloadURL) este returnat drept răspuns.Codul din `MainController` compară această versiune cu cea instalată în prezent pe computerul utilizatorului. Un buton care permite utilizatorului să deschidă adresa URL de descărcare în browser-ul default al sistemului de operare este activat în interfața grafică odată ce diferența dintre versiuni fost verificată.

Interogările de actualizare sunt executate în fire de execuție dedicate folosind clasa /textitTasks specifice Java, prevenind blocarea firului principal al aplicației pentru a preveni potențiale neplaceri vizuale. Instrucțiunile **`process.waitFor()`** și **`process.destroy()`** sunt utilizate pentru a guverna scriptul extern, asigurându-se că execuția este finalizată și că resursele sunt eliberate corect.

5.2 Selectarea driverelor Windows ce necesită actualizări

Aplicația dezvoltată de mine în acest proiect încorporează o tehnică automată pentru identificarea și prezentarea driverelor Microsoft sau de sistem prezente pe computerul utilizatorului, care trebuie actualizate. Tehnologiile native **`PowerShell`**[11] sunt combinate cu un mediu grafic interactiv creat cu *JavaFX* pentru a realiza această funcționalitate. Obiectivul principal al acestei componente este de a oferi utilizatorului o modalitate simplă, dar fiabilă, de a detecta și rezolva problemele legate de versiunile de drivere învechite.

Apelul către sistemul Windows Update prin intermediul unui script `PowerShell` integrat constituie fundamentul acestei funcționalități. Doar actualizările de drivere accesibile prin canalul oficial Microsoft sunt solicitate în mod specific de **`Get-WindowsUpdate`**. După aceea, fiecare rezultat este procesat iterativ pentru a extrage întregul titlul driver-ului, care include adesea numele componentei hardware, precum și versiunea curentă. Prin crearea unui proces nativ în sistemul de operare, aplicația permite executarea asincronă a comenzilor `PowerShell`[11].

Odată ce driverele care au nevoie de update au fost colectate, aplicația realizează o analiză structurală a șirurilor de caractere rezultate. Sunt definite două metode auxiliare

- Metoda **`extractTitle()`** - care izolează doar numele driver-ului sau al componentei dedicate driver-ului din întreg textul extras.
- Metoda **`extractVersion`** - care face acelasi lucru, doar ca pentru versiunea curenta a driver-ului.

O structură `Map<Integer, List<String>>` conține rezultatul, fiecare cheie indicând un identificator incremental unic, iar valoarea corespunzătoare fiind o listă de două elemente: versiunea sugerată și numele driverului. Acest tip de organizare simplifică manipularea ulterioară a datelor, inclusiv afișarea lor într-o interfață grafică cu utilizatorul.

Capacitatea acestui cod de a crea linkuri oficiale către `Microsoft-Update-Catalogue`[2], în loc să intermedieze descărcarea directă a driverelor, este una dintre cele mai importante caracteristici ale sale. Această alegere de design elimină posibilitatea descărcării din surse dubioase sau nesigure.

Linkul este creat dinamic prin concatenarea numelui driverului - care a fost deja extras - cu o adresă URL de căutare existentă. Pentru a se asigura că utilizatorul vede mai întâi cea mai recentă versiune, această adresă URL este preconfigurată pentru a sorta rezultatele în funcție de data publicării (`scol=DateComputed.sdir=desc`).

Partea interfeței grafice a aplicației care afișează driverele Microsoft ce trebuie actualizate este construită dinamic utilizând un design scalabil și modular, bazat pe componente specifice *JavaFX*.

Mai exact, metodele **`addDriverEntriesFromMap()`** și **`addUpdateEntry()`** ale clasei `MainController` realizează logică de populare vizuală a structurii grafice `driverUpdatesEntries`. Pe lângă un buton de interacțiune care inițiază descărcarea actualizării asociate în browserul implicit al computerului, fiecare intrare creată și afișată are etichete text (`Label`) care arată numele driverului și versiunea instalată în prezent.

Cu actualizări de conținut în timp real care depind de rezultatul procesării asincrone efectuate în firul secundar de execuție, această paradigmă de construcție a interfeței dinamice păstrează răspunsul și consecvența aplicației, permițând în același timp o reprezentare elegantă, interactivă și transparentă a actualizărilor disponibile.

[2]Microsoft-Update-Catalogue: https://www.catalog.update.microsoft.com/Home.aspx

Capitolul 6

Interfața grafică

Am ales să utilizez framework-ul **`JavaFX`**[2] pentru a crea interfața grafică a programului de mentenanță pe care l-am creat, o decizie care a fost susținută atât din punct de vedere arhitectural, cât și tehnologic. Prin integrarea completă a sistemului de fișiere *`FXML`*, a stilurilor *`CSS`* și a legăturilor reactive, *JavaFX* oferă o soluție de ultimă generație, modulară și extensibilă pentru crearea de interfețe grafice cu utilizatorul. Acest framework s-a dovedit a fi opțiunea perfectă pentru o aplicație care încorporează o mulțime de activități sistemice în fundal, afișarea datelor în timp real și actualizări de stare datorită capacității sale de a gestiona eficient interfețe cu utilizatorul complicate, componente vizuale dinamice și interacțiuni asincrone. Simultan, integrarea naturala cu codul *Java* permite o comunicare cu partea de back-end fiabilă și rapidă.

Interfața este structurată modular în jurul a 3 tab-uri principale: *Cleaner, Performance* și *Updates*, fiecare oferind utilizatorului posibilitatea dea pune în acțiune funcționalitățile descrise până acum în cadrul lucrării, și o fereastră popup pentru acces rapid.

6.1 Panoul *Cleaner*

Curățarea și optimizarea fișierelor tranzitorii, inutile sau redundante apărute ca urmare a erorilor este punctul central al tab-ului *Cleaner*. Interfața permite utilizatorului să aleagă sau să filtreze tipurile de fișiere pentru analiză și ștergere, prezentând mai multe secțiuni verticale aranjate într-un `VBox`. După scanare, fiecare secțiune este actualizată constant cu detalii complete despre fișierele găsite.

Prin utilizarea casetelor de selectare pentru a selecta categoriile de fișiere temporare și a etichetelor elegante pentru a vizualiza volumele estimate, secțiunea *Junk Files* este concepută ca un panou cu diverse componente vizuale și interactive. În plus, interfața are două butoane principale, `SCAN` și `CLEAR`, care pornesc și termină procesele de analiză, respectiv curățare. Componentele sunt aranjate într-un `VBox` care are `HBox-uri` pentru acțiuni și mesaje de stare, precum și numeroase `GridPane`-uri pentru afișarea grafică a

pictogramelor și a datelor de stocare.

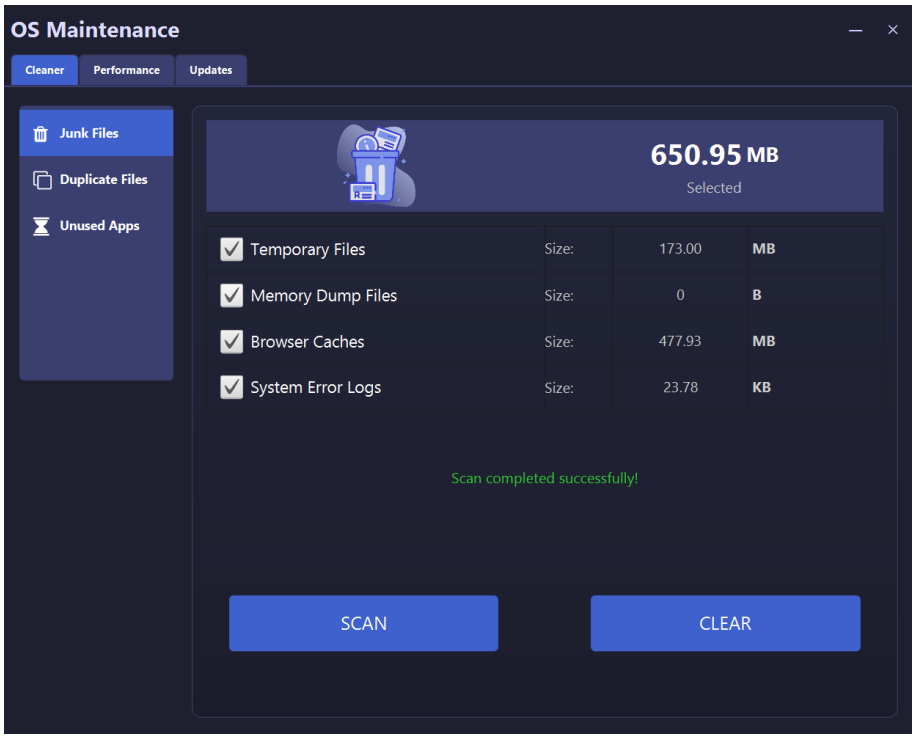

Figura 6.1: Panoul Cleaner, secțiunea JunkFiles

Subsecțiunile *Unused Apps* (*Figura 6.3*) și *Duplicates Files* (*Figura 6.2*), pe de altă parte, permit să alegi un director de lucru și afișează rezultatele într-un panou de derulare vertical, umplut dinamic cu intrări `VBox`. Când utilizatorul începe o scanare, sistemul afișează rezultatele filtrate împreună cu opțiunile de acțiune (cum ar fi ștergerea sau descărcarea), ceea ce reprezintă un exemplu de comportament *master-detail*. O interacțiune intuitivă și fiabilă este posibilă datorită acestei structuri, care se bazează pe machete fluide și este gestionată dinamic din backend (`MainController`) prin legături *JavaFX* și rutine de gestionare a evenimentelor. Front-end-ul (*FXML*) și back-end-ul (controlerul *Java*) lucrează împreună pentru a crea o interfață responsivă, scalabilă și extensibilă în această implementare, care demonstrează un design modern.

Cunoscut programatic sub numele de `unusedAppsPane`, panoul aplicațiilor neutilizate este conceput pentru a facilita găsirea și gestionarea fisierelor care nu au fost utilizate de mult timp sau care ocupă spațiu pe disc inutil. Arhitectură acestui panou se bazează pe un `VBox` principal care are trei secțiuni principale: un `ScrollPane` care va afișa rezultatele scanării într-un mod scalabil și derulabil, un `HBox` care va afișa directorul selectat și un `GridPane` superior pentru controlul inițial.

Utilizatorul poate selecta un director de pe disc folosind butonul din secțiunea de tip grid a acestui panou, care va fi apoi examinat pentru a identifica aplicațiile instalate sau fișierele executabile care nu au fost accesate recent. Această alegere este crucială deoarece definește zona în care poate fi aplicată analiză, restricționând contextul folderului

respectiv, ceea ce maximizează performanța și reduce posibilitatea unor rezultate fals pozitive.

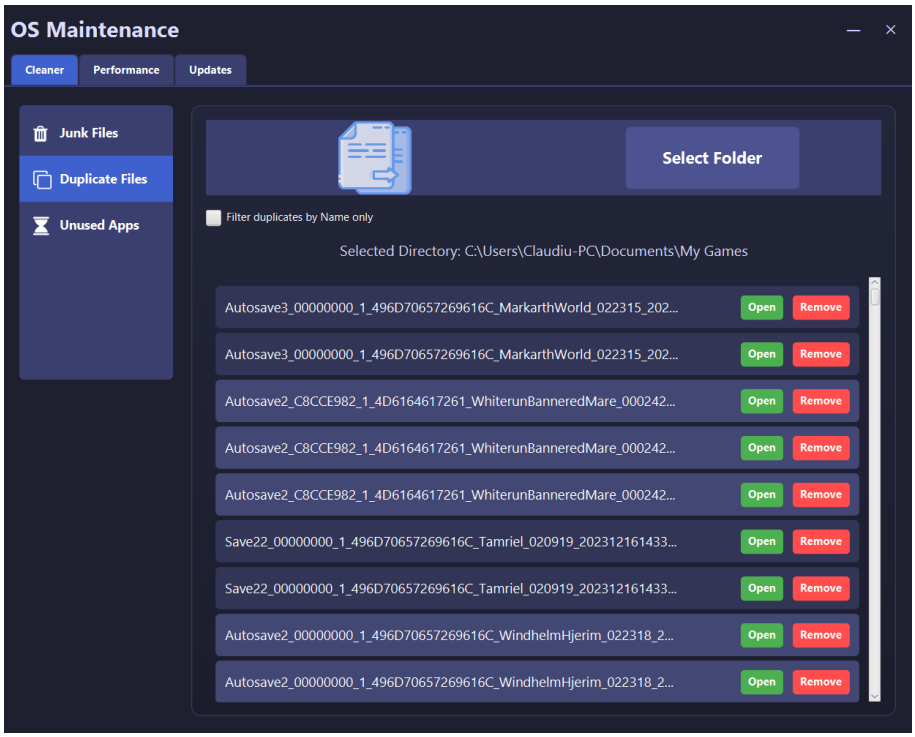

Figura 6.2: Panoul Cleaner, secțiunea Duplicate Files

Prin urmare, tab- ul *Cleaner* gestiona atât ștergerea fișierelor nefolositoare,cât și cazuri complicate cu sute de fișiere duplicate sau inutile. Interfața este concepută pentru a susține conceptele de modularitate, reutilizare și divizare a responsabilităților - toate acestea fiind cruciale pentru crearea de aplicații desktop contemporane.

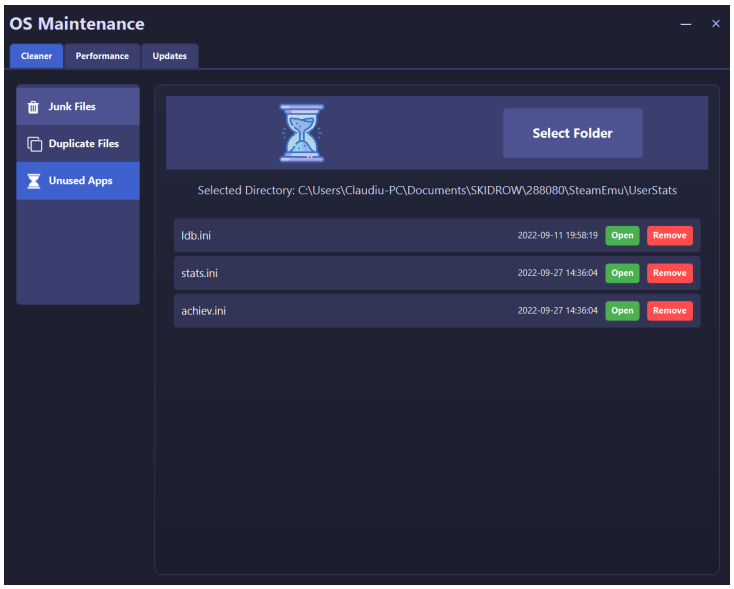

Figura 6.3: Panoul Cleaner, secțiunea Unused Files

6.2 Panoul *Performance*

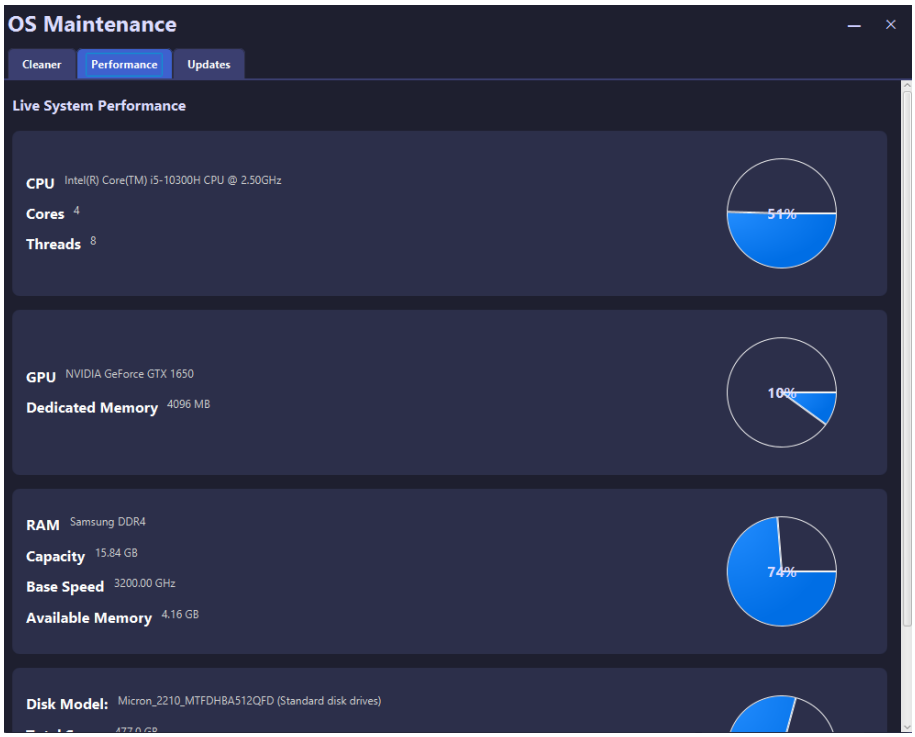

Figura 6.4: Panoul Performance

Scopul tab-ului *Performance* este de a urmări cât de mult procesor, RAM, placă video și spațiu pe disc sunt utilizate în timp real. O diagramă circulară animată[10] cu o etichetă centrală care arată procentul de utilizare este utilizată pentru a reprezenta fiecare dintre aceste elemente. Executorii programați (`ScheduledExecutorService`) care utilizează biblioteca OSHI sau scripturi externe (cum ar fi Python pentru GPU) pentru a verifica starea sistemului actualizează aceste grafice în mod regulat (la fiecare 5 secunde). În plus, etichetele adiacente afișează informații descriptive (cum ar fi numele GPU și frecvența RAM).

Fiecare secțiune este izolată și poate fi verificată independent datorită designului modular, fiind incorporate intr-un container de tip `ScrollPane` cu politici de scroll configurate la `fitToWidth` și `fitToHeight`. Pe lângă faptul că oferă o afișare clară și ușor de utilizat a stării sistemului în timp real, modularitatea și extensibilitatea acestei arhitecturi permit extinderea viitoare cu metrici suplimentare.

6.3 Panoul *Updates*

Tab-ul *Updates* este împărțită în două secțiuni: una pentru identificarea driverelor generice Windows învechite și o alta pentru actualizarea driverelor plăcii video (exclusiv *NVIDIA*, folosind o interogare API și extragere `XML/JSON`). În ceea ce privește driverele

NVIDIA, aplicația extrage imediat numele plăcii video, verifică versiunea instalată local cu versiunea de pe site-ul oficial și, dacă este necesar, oferă un buton pentru descărcare din browser. Fiecare driver descoperit este afișat în interfață într-un container (`HBox`) cu informații și un buton de descărcare care este adăugat dinamic în `VBox`-ul principal (`driverUpdatesEntries`).

Având în vedere toate aceste aspecte, tab-ul *Updates* este un exemplu excelent al utilizării cu succes a principiilor interfeței reactive în *JavaFX*, oferind un aspect ușor de utilizat care facilitează interacțiunea fără probleme a utilizatorului cu sistemul de actualizare a driverelor.

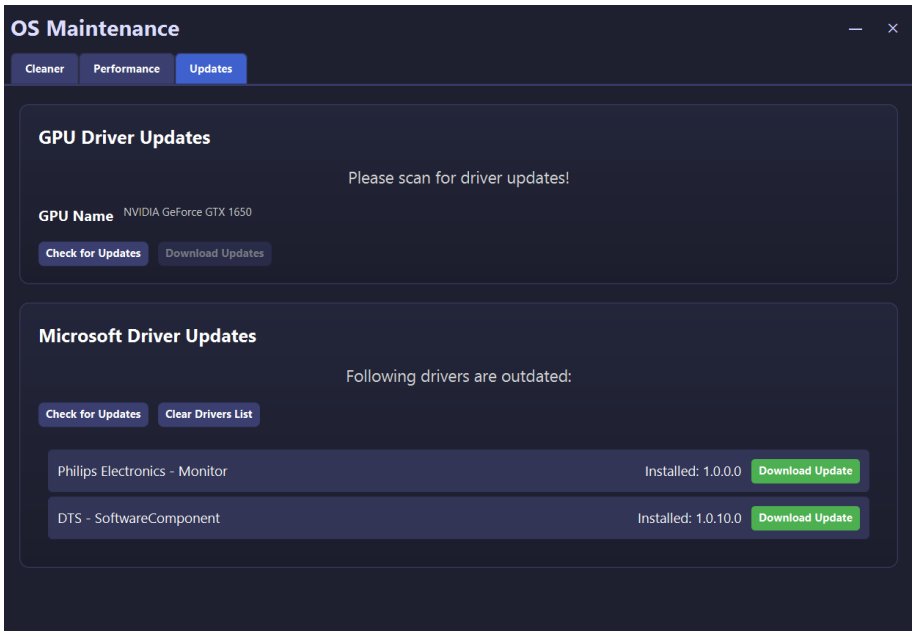

Figura 6.5: Panoul Updates cu drivere învechite listate

6.4 Fereastra pop-up pentru acces rapid

Creata pentru access rapid la uneltele aplicației principale când este închisa.

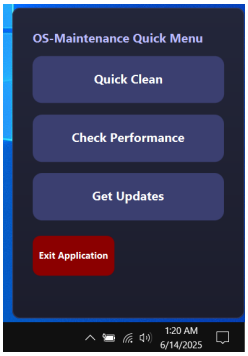

Figura 6.6: Enter Caption

Capitolul 7

Concluzii

7.1 Perspective și îmbunătățiri

Cu toate că aplicația atinge un nivel funcțional complet și este stabilă în utilizare, există numeroase direcții prin care această poate fi extinsă. În primul rând, un sistem de rapoarte automate pe baza performanței sistemului între sesiunile de clean-up ar permite o analiză predictivă a eficienței oferite de aplicație în timp.

De asemenea , și modulul de actualizare ar putea fii îmbunătățit prin programarea, descărcarea și instalarea automată a driverelor, fara a mai fii nevoie de intervenția utilizatorului, dar și prin extinderea suportului de căutare și descărcare a driverelor grafice și pentru alți producători în afara de NVIDIA.

Nu în ultimul rând , alt pas crucial în scalare și îmbunătățire îl reprezintă portarea aplicației și către alte platform non-Windows, precum distribuții Linux sau chiar platforme mobile Android.

7.2 Concluzie finală

Dezvoltarea acestui tool de mentenanța a sistemului de operare Windows a necesitat o abordare interdisciplinară, combinând concepte din ingineria aplicațiilor și tehnologiilor software cu aspecte de ultimă generație din domeniul sistemelor de operare. Scopul principal al dezvoltării, pe care l-am atins cu ajutorul cunoștințelor adunate în timpul masterului, a fost de a cerceta și implementa o soluție fiabilă, eficientă și ușor de utilizat, care să ofere funcții tangibile utilizatorilor obișnuiți.

În concluzie, în cadrul acestei lucrări am prezentat amănunțit aplicația creată de mine, evidențiind beneficiul tehnologic adus de catre aceasta, dar și modul în care îmbină eficient funcții sofisticate de scanare, curățare, diagnosticare și actualizare, cu o interfață grafică intuitivă și modernă, oferind utilizatorilor Windows o soluție unică de întreținere a sistemului lor de operare și prelungirea performanței acestuia pe cat mai mult timp.

Bibliografie

[1] Andrea Allievi, Alex Ionescu, Mark E. Russinovich și David A. Solomon, *Windows® Internals 7th Edition, Book 2*, Microsoft Press, 2021, URL: https://www.microsoftpressstore.com/store/windows-internals-part-2-9780135462409.

[2] Baeldung, „*Introduction to JavaFx*", în *Spring Courses - Baeldung* (2024), URL: https://www.baeldung.com/javafx.

[3] Yatin Batra, „*Getting Started with OSHI*", în *Core Java* (2025), URL: https://www.javacodegeeks.com/getting-started-with-oshi.html.

[4] Vic Broquard, *Windows MFC Programming I 3rd edition*, Broquard eBooks, 2014, URL: https://www.amazon.com/Windows-MFC-Programming-Vic-Broquard/dp/1941415571.

[5] Andrew Cron și Wes McKinney, „*gpustats: GPU Library for Statistical Computing in Python*", în *Computer architecture - GPU* (2011), URL: https://www.researchgate.net/publication/340142729_gpustats_GPU_Library_for_Statistical_Computing_in_Python.

[6] Carl Dea, *JavaFX 2.0: Introduction by Example*, Apress, 2011, URL: https://link.springer.com/book/10.1007/978-1-4302-4258-1.

[7] Brian Gotez, Tim Peierls, Joshua Bloch, Joseph Bowbeer, David Holmes și Doug Lea, *Java Concurency in Practice*, Addyson-Wesley, 2006, URL: https://mrce.in/ebooks/Java%20Concurrency%20in%20Practice.pdf.

[8] Steph Howson, *Python XML Tutorial with ElementTree: Beginner's Guide*, 2024, URL: https://www.datacamp.com/tutorial/python-xml-elementtree.

[9] Ryan Mitchell, *Web Scraping with Python: Collecting More Data from the Modern Web*, O'REILLY, 2018, URL: https://www.amazon.com/Web-Scraping-Python-Collecting-Modern/dp/1491985577.

[10] Java Oracle, *Introduction to JavaFX Animations*, 2024, URL: https://dev.java/learn/javafx-animations/.

[11] Harm Veenstra, „*Using the PowerShell PSWindowsUpdate module*", în *PowerShell Security* (2024), URL: https://powershellisfun.com/2024/01/19/using-the-powershell-pswindowsupdate-module/.